



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2233 Programación Avanzada (2026-1)

Tarea 3

Entrega

- Tarea y README.md
 - Fecha y hora oficial: 19 de junio 20:00 hrs.
 - Lugar: Repositorio personal de GitHub — Carpeta: Tareas/T3/.
El código debe estar en la rama (*branch*) por defecto del repositorio: *main*.
 - Pauta de corrección: [en este enlace](#).
 - Bases generales de tareas (descuentos): [en este enlace](#).
- Ejecución de tarea: La tarea será ejecutada **únicamente** desde la terminal del computador. Además, durante el proceso de corrección, se cambiará el nombre de la carpeta “T3/” por otro nombre y se ubicará la terminal dentro de dicha carpeta antes de ejecutar la tarea. **Los *paths* relativos utilizados en la tarea deben ser coherentes con esta instrucción.**

Objetivos

- Utilizar conceptos de interfaces y PyQt6 para implementar una aplicación gráfica e interactiva.
- Traspasar decisiones de diseño y modelación en base a un documento de requisitos.
- Diseñar e implementar una arquitectura cliente-servidor. Además, en el cliente se debe entender y aplicar los conceptos de *back-end* y *front-end*.
- Aplicar conocimientos de *threading* en interfaces (Thread y/o QThread).
- Aplicar conocimientos de señales.
- Aplicar conocimientos de creación de redes para comunicación efectiva entre computadores (*networking*).
- Aplicar conocimientos de manejo de *bytes* para seguir un protocolo de comunicación preestablecido.

Índice

1. <i>DCCuestionados</i>	3
1.1. Introducción	3
2. Flujo del programa	4
3. Mecánicas de Juego	5
3.1. Sesión del Jugador	5
3.1.1. Registro e inicio de sesión	5
3.1.2. Perfil e historial	5
3.2. Categorías	5
3.3. Ronda de Juego	5
3.3.1. Flujo de una ronda	5
3.3.2. Preguntas	6
3.3.3. Comodines	6
3.3.4. Finalizar Partida	7
3.4. Sistema de Puntaje	8
3.4.1. Puntaje por pregunta correcta	8
3.4.2. Puntaje récord	9
3.4.3. Respuesta incorrecta o tiempo agotado	9
3.4.4. Derrota	9
3.5. Leaderboard	9
4. Interfaz Gráfica e interacción	10
4.1. Ventana de Inicio de sesión	10
4.2. Ventana principal	11
4.3. Ventana de preguntas	12
4.4. Ventana de Fin de partida	13
5. Arquitectura cliente-servidor	14
5.1. Serialización	14
5.2. API externa	15
5.2.1. Elección y requisitos de la API externa	15
5.2.2. Documentación obligatoria en el README	15
5.2.3. Política de consumo y almacenamiento local	16
5.3. Concurrencia y Gestión de Conexiones	16
5.3.1. Multi-threading	16
5.3.2. Sincronización y Seguridad de Hilos	16
5.3.3. Protocolo de Desconexión Robusta y Manejo de Errores	16
6. <i>WebServices</i>	17
7. <i>Base de datos</i>	20
8. Archivos	21
8.1. <code>parametros.py</code>	21
9. <code>.gitignore</code>	22
10. Importante: Corrección de la tarea	23
11. Restricciones y alcances	23

1. *DCCuestionados*

1.1. Introducción

Era el día del examen final de Programación Avanzada. Saliste convencido de que te había ido bien, y con una sonrisa te diriges a tu casa. El sol de San Joaquín pegaba justo en la cara cuando cruzaste la puerta del edificio. Cerraste los ojos un segundo para disfrutarlo. Fue exactamente en ese segundo cuando sentiste la bolsa en la cabeza. Te subieron a algo que olía a café frío, a gato chico y a papel de impresora barata, y que se movió durante exactamente el tiempo suficiente para que perdieras toda noción de hacia dónde ibas. Solo se escuchaba, muy de fondo, lo que juraste era la canción de entrada de un programa de televisión. Cuando la bolsa cayó, entendiste todo y nada al mismo tiempo.

Cada alumno que rinde el examen de Programación Avanzada es interceptado al salir, equipado con un micrófono de solapa, conectado en vivo a una señal satelital que se transmite a 47 países y sometido al ***DCCuestionados***: el único programa de televisión donde tu coeficiente académico, tu dignidad y tu futuro profesional dependen exclusivamente de cuánto sabes responder bajo presión y con el tiempo en tu contra. El show tiene patrocinadores. Tiene fanáticos. Tiene comentaristas que analizan cada respuesta en tiempo real. En algunos países, los episodios se doblan al idioma local.

Nadie sabe exactamente cuándo los ayudantes del DCC se cansaron del sistema tradicional de evaluación. Algunos dicen que fue después del semestre en que el 70 % del curso entregó la misma tarea con distintos nombres de variables. Otros apuntan al día en que alguien preguntó en discussions si Python era un animal. Hay quienes culpan al alumno que entregó un archivo `main.py` completamente vacío con un comentario que decía `## Lo siento, como IA no puedo completar esta tarea por ti`. Pero la teoría más extendida es también la más simple: los ayudantes estaban agotados de sus propios ramos (algunas teorías locas mencionan estructuras de datos), de sus propias tareas, de sus propias interrogaciones a las 8 AM, y en algún punto entre el tercer café en vena de la madrugada y la corrección número cuarenta de la misma función mal implementada, decidieron hacer un programa de entretenimiento a nivel mundial que evaluara el nivel de conocimiento de cada uno de sus estudiantes. Lo que empezó como una broma en el canal del equipo docente terminó siendo el show más visto de la televisión universitaria.

Ahora eras tú el que estaba sentado bajo los focos, con el micrófono de solapa ya prendido, tu cara transmitida en vivo en el monitor gigante instalado frente a una audiencia en directo. Los ayudantes, perfectamente uniformados con la camiseta del DCC, operaban las cámaras con la misma seriedad con que alguna vez revisaron tus tareas. El animador te extendió la mano con una sonrisa perfectamente ensayada y dijo, con la entonación de quien lleva años diciéndolo: *«Bienvenido al DCCuestionados. El mundo está listo. ¿Lo estás tú?»*

2. Flujo del programa

DCCuestionados es un juego de trivia *single player* que tiene como objetivo principal responder de forma correcta la mayor cantidad de preguntas consecutivas, cada una en el menor tiempo posible, desafiando al jugador a mantener su racha, pues una pregunta incorrecta significa el fin de la partida.

El funcionamiento de la aplicación comienza con la ejecución del servidor, el cual debe quedar en estado de escucha para gestionar las solicitudes de los usuarios y actuar como intermediario con la [API externa](#) de trivia. Una vez que el servidor se encuentra activo, se debe iniciar el programa del cliente, el cual establecerá una conexión al servidor. Al conectarse exitosamente, el usuario irá a la [Ventana de Inicio de sesión](#), donde deberá ingresar un nombre de usuario y una contraseña. Estos datos serán enviados al servidor para ser validados. Si el nombre de usuario ya se encuentra registrado en el sistema, el servidor verificará que la contraseña coincida para permitir el ingreso. En caso de que el nombre de usuario no exista, se verificará que este sea alfanumérico, y si lo es, el sistema lo registrará, asociándole automáticamente la contraseña ingresada. Adicionalmente, el servidor verifica que no exista una sesión activa para ese mismo nombre de usuario; si ya hay una sesión abierta con ese `username`, el ingreso es rechazado (ver detalle en [Registro e inicio de sesión](#)). Tras una validación exitosa, el sistema permitirá el acceso al menú principal o *Lobby*, que corresponde a la [Ventana principal](#).

Una vez en la Ventana Principal, el usuario podrá visualizar las distintas categorías de trivia disponibles y consultar el *leaderboard* con los puntajes más altos obtenidos por otros jugadores. Para desplegar esta información, el cliente realizará una solicitud al servidor, quien se encargará de obtener los datos desde la API externa. Es fundamental recordar que el cliente no debe tener comunicación directa con la API, todas las peticiones deben ser gestionadas y filtradas por el servidor. En esta interfaz, el jugador podrá seleccionar la temática de su interés y, al confirmar su elección, el sistema gatillará la transición hacia la Ventana de Juego, solicitando al servidor el *batch* de preguntas acorde a la categoría elegida, para comenzar la partida.

Al ingresar a la Ventana de Juego, el cliente recibirá el *batch* de preguntas enviado por el servidor. El jugador deberá responder cada una de forma secuencial, visualizando una sola pregunta a la vez en la interfaz de [Ventana de preguntas](#). Cada pregunta estará sujeta a un tiempo máximo para responder, el cual se debe mostrar en la interfaz gráfica. Cada vez que se seleccione una opción, la respuesta será enviada al servidor para su validación inmediata. El puntaje se calculará de forma dinámica en el servidor, considerando la rapidez del jugador al contestar: entre menor sea el tiempo de respuesta, mayor será la bonificación obtenida. La partida finalizará si el jugador responde todas las preguntas correctamente, considerándose este caso como una victoria. Si, por otro lado, el usuario responde una pregunta de manera incorrecta, o si el temporizador llega a cero antes de ingresar una respuesta, la partida también finalizará instantáneamente, pero se considerará una derrota. En ambos casos, en cuanto se finaliza la partida, el servidor procesará el puntaje final, lo registrará en la base de datos y notificará al cliente el término del juego. Al recibir esta notificación, la aplicación del cliente deberá gatillar la transición hacia la [Ventana de Fin de partida](#), desplegando el resumen completo con el puntaje total obtenido, la cantidad de respuestas correctas y la duración total de la sesión. Desde esta interfaz, el jugador podrá revisar sus resultados y podrá regresar al *Lobby* presionando un botón de Volver al inicio, donde podrá visualizar su posición actualizada en el ranking global y por categorías.

3. Mecánicas de Juego

3.1. Sesión del Jugador

3.1.1. Registro e inicio de sesión

Cada jugador accede al sistema mediante un **nombre de usuario** y una **contraseña**.

- **Registro e inicio de sesión:** El jugador ingresa un nombre de usuario y una contraseña en un único flujo. Si el nombre de usuario no existe en la base de datos, la cuenta se crea automáticamente y el jugador accede al sistema de inmediato. Si el nombre de usuario ya existe, se valida la contraseña; si es incorrecta, el acceso queda denegado y se notifica al jugador. No pueden coexistir dos jugadores con el mismo nombre de usuario conectados al mismo tiempo; si el usuario ya tiene una sesión activa, el nuevo intento de acceso queda denegado hasta que la sesión anterior finalice. La contraseña **no se almacena en texto plano**; el servidor guarda únicamente su hash.

3.1.2. Perfil e historial

Al registrarse, cada jugador **nuevo** parte con un puntaje acumulado igual a 0. El **puntaje acumulado** corresponde a la suma de todos los puntajes finales obtenidos en cada partida jugada; es decir, al finalizar cada partida, el puntaje obtenido en ella se suma al puntaje acumulado del jugador, independientemente de si la partida terminó en victoria o derrota. Los jugadores recurrentes conservan su historial completo: puntaje acumulado total, victorias, derrotas y cantidad de partidas jugadas. Adicionalmente, el puntaje acumulado se registra **por categoría**, de modo que el sistema mantiene un historial independiente del desempeño del jugador en cada categoría de preguntas.

3.2. Categorías

El juego cuenta con **NUMERO_CATEGORIAS** categorías de preguntas disponibles. Antes de iniciar cada partida, el jugador debe seleccionar una de ellas; todas las preguntas de esa partida pertenecerán exclusivamente a la categoría elegida. Las categorías disponibles son entregadas por el servidor al cliente al momento de la selección.

3.3. Ronda de Juego

3.3.1. Flujo de una ronda

1. **Selección de categoría:** Antes de iniciar cada partida, el jugador debe seleccionar una de las **NUMERO_CATEGORIAS** categorías disponibles. Todas las preguntas durante esa partida pertenecerán a la categoría elegida.
2. **Inicio de partida:** Una vez seleccionada la categoría, la partida comienza de inmediato. Cada partida contiene como máximo **NUMERO_PREGUNTAS** preguntas. Al inicio de la partida, el jugador recibe un comodín de cada tipo disponible, cada uno utilizable **a lo más una vez** durante la partida.
3. **Preguntas:** Las preguntas se presentan de forma secuencial. Cada pregunta es de **selección múltiple** con **NUMERO_ALTERNATIVAS** alternativas, de las cuales exactamente una es correcta. El jugador dispone de **TIEMPO_POR_PREGUNTA** segundos para seleccionar una respuesta. Si el tiempo expira sin que el jugador haya respondido, la pregunta se contabiliza como incorrecta. La partida finaliza en cualquiera de los siguientes casos:
 - El jugador completa las **NUMERO_PREGUNTAS** preguntas (*victoria*).
 - El jugador responde incorrectamente una pregunta (*derrota*).

- El tiempo expira sin que el jugador haya respondido (*derrota*).
- El jugador falla ambos intentos al usar el comodín *Doble Oportunidad* (*derrota*). Ver [Comodines](#).

Para más información revisar [Finalizar Partida](#).

4. **Resultados:** Al finalizar la partida, se despliega una ventana con el puntaje final obtenido, el número de preguntas respondidas correctamente y el tiempo total de duración de la partida, junto con un botón para **volver al inicio**. El puntaje obtenido se registra automáticamente en la base de datos.

3.3.2. Preguntas

Cada pregunta presentará exactamente [NUMERO_ALTERNATIVAS](#) alternativas de respuesta, de las cuales solo una es correcta. Las alternativas se muestran en orden aleatorio en cada presentación. El jugador selecciona su respuesta haciendo clic sobre la alternativa deseada. El jugador **no puede conocer** cuál es la respuesta correcta antes de haber respondido o de que expire el tiempo.

Cada jugador dispone de [TIEMPO_POR_PREGUNTA](#) segundos para responder. Una vez que el jugador selecciona una alternativa, hay un tiempo de espera de [TIEMPO_RESPUESTA](#) antes de que se muestre en pantalla si la respuesta es correcta o incorrecta. Cuando se muestre el resultado, este permanece visible durante [TIEMPO_MOSTRAR_RESPUESTA](#) segundos antes de pasar a la siguiente pregunta.

Cada pregunta otorga un puntaje P_{pregunta} si se responde correctamente. En caso contrario, se descuenta al puntaje del jugador [PENALIZACION_INCORRECTA](#) puntos y la partida **finaliza inmediatamente**, salvo cuando el jugador haya activado el comodín *Doble Oportunidad* (ver [Comodines](#)).

3.3.3. Comodines

Al inicio de cada partida, el jugador recibe los siguientes comodines. Cada comodín puede usarse **a lo más una vez** por partida. El uso de uno o más comodines en una pregunta reduce el puntaje base de dicha pregunta según las fórmulas detalladas más adelante en [Puntaje por pregunta correcta](#)

- **Segunda Oportunidad** (*Second Chance*): Permite responder dos veces a una misma pregunta. Al activar este comodín, este se consume independientemente del resultado; es decir, **si el jugador acierta en el primer intento**, el puntaje se calcula aplicando [FACTOR_PENALIZACION_COMODIN](#) sobre el puntaje base, pero el contador de racha **sigue avanzando con normalidad**. Si el jugador falla en el primer intento, se aplica [PENALIZACION_INCORRECTA](#) y el **contador de racha se reinicia a cero**, pero la partida no termina, pudiendo seleccionar otra alternativa. Si acierta en el segundo intento, el puntaje se calcula de forma habitual con el t_{restante} disponible en ese momento, incluyendo el bono de velocidad. Si también falla en el segundo intento, se aplica nuevamente [PENALIZACION_INCORRECTA](#) y la partida finaliza. El tiempo sigue corriendo durante ambos intentos sin reiniciarse. **Este comodín no tiene efecto si el tiempo se agota antes de que el jugador seleccione una respuesta**; en ese caso, la pregunta se contabiliza directamente como incorrecta y la partida finaliza, sin dar lugar a un segundo intento.
- **Cambio de Pregunta:** Reemplaza la pregunta actual por una nueva pregunta distinta de la misma categoría. La pregunta original se **descarta definitivamente** y no volverá a aparecer en la partida. Al activar este comodín, el temporizador **se reinicia** al valor máximo [TIEMPO_POR_PREGUNTA](#), permitiendo al jugador disponer del tiempo completo para responder la nueva pregunta. Sin embargo, como compensación por el reinicio del tiempo, la nueva pregunta **no otorgará bono de velocidad** ($B_{\text{velocidad}} = 0$) independientemente de cuán rápido se responda. El puntaje base seguirá siendo penalizado por el uso del comodín de la forma habitual.

- **Eliminar Opciones (50/50):** Descarta la mitad de las alternativas incorrectas (redondeando hacia abajo), dejando al jugador con la alternativa correcta y un número reducido de distractores. El tiempo sigue corriendo normalmente y el bono de velocidad se mantiene. Sin embargo, la penalización sobre el puntaje base es proporcional al número de alternativas descartadas n :

$$P'_{\text{base}} = \lfloor P_{\text{base}} \cdot \text{FACTOR_PENALIZACION_COMODIN}^n \rfloor$$

donde n es el número de alternativas incorrectas eliminadas.

En una misma pregunta se pueden ocupar diferentes comodines. La siguiente tabla resume las combinaciones de comodines permitidas en una misma pregunta según el orden de activación. Las filas representan el **primer comodín** activado y las columnas el **segundo**. La única combinación permitida es activar primero *Cambio de Pregunta* y luego *50/50*, ya que tiene sentido estratégico: primero se obtiene una nueva pregunta y luego se eliminan alternativas de ella. El orden inverso (*50/50* seguido de *Cambio de Pregunta*) no está permitido, pues el cambio de pregunta descartaría la pregunta sobre la que ya se aplicó el *50/50*, haciendo el primer comodín completamente inútil.

		Segundo comodín		
		Doble Oportunidad	Cambio de Pregunta	50/50
Primer comodín	Doble Oportunidad	—	×	×
	Cambio de Pregunta	×	—	✓
	50/50	×	×	—

Cuadro 1: Matriz de combinabilidad de comodines según orden de activación.

El puntaje base efectivo P'_{base} depende del comodín o combinación de comodines utilizados en la pregunta, esto se vera en la sección [Sistema de Puntaje](#)

3.3.4. Finalizar Partida

Una partida finaliza si y solo si se cumple alguna de las siguientes condiciones:

1. El jugador selecciona una alternativa incorrecta (salvo que esté activo el comodín *Doble Oportunidad*).
2. El tiempo de [TIEMPO_POR_PREGUNTA](#) se agota sin que el jugador haya respondido.
3. El jugador falla ambos intentos al usar *Doble Oportunidad*.
4. Se completó el número máximo de preguntas [NUMERO_PREGUNTAS](#).

Una vez finalizada la partida, se despliega una ventana con el puntaje total obtenido, el número de preguntas respondidas correctamente y el tiempo total de duración de la partida, junto con un botón para **volver al inicio**. El puntaje se registra en la base de datos

3.4. Sistema de Puntaje

3.4.1. Puntaje por pregunta correcta

Al responder correctamente, el jugador obtiene un puntaje compuesto por tres términos:

$$P_{\text{pregunta}} = P'_{\text{base}} + B_{\text{velocidad}} + B_{\text{racha}}$$

donde P'_{base} es el puntaje base *posiblemente reducido* por el uso de un comodín en esa pregunta (ver [Comodines](#)), y B_{racha} es igual a [BONUS_RACHA](#) únicamente cuando el contador de respuestas correctas de la partida alcanza un múltiplo exacto de [NUMERO_DE_RACHA](#), siendo $B_{\text{racha}} = 0$ en cualquier otro caso.

Puntaje base. Valor fijo otorgado por cualquier respuesta correcta, independiente del tiempo empleado:

$$P_{\text{base}} = \text{PUNTAJE_BASE_CORRECTA}$$

El puntaje base efectivo P'_{base} depende del comodín o combinación utilizada en la pregunta (ver [Comodines](#)):

- **Sin comodín:**

$$P'_{\text{base}} = P_{\text{base}}$$

- **Solo Doble Oportunidad o solo Cambio de Pregunta:**

$$P'_{\text{base}} = \lfloor P_{\text{base}} \cdot \text{FACTOR_PENALIZACION_COMODIN} \rfloor$$

- **Solo 50/50**, donde n es el número de alternativas incorrectas eliminadas:

$$P'_{\text{base}} = \lfloor P_{\text{base}} \cdot \text{FACTOR_PENALIZACION_COMODIN}^n \rfloor$$

- **Cambio de Pregunta + 50/50**, donde n es el número de alternativas incorrectas eliminadas:

$$P'_{\text{base}} = \lfloor P_{\text{base}} \cdot \text{FACTOR_PENALIZACION_COMODIN}^{n+1} \rfloor$$

En todos los casos, si P'_{base} resulta negativo por efecto de las penalizaciones, se considera igual a 0.

Bono de velocidad. Recompensa al jugador por responder rápidamente. Decece de forma lineal desde [BONUS_VELOCIDAD_MAX](#) al responder de forma instantánea, hasta 0 al agotar el tiempo. El tiempo restante t_{restante} es medido y validado por el **servidor** al momento de recibir la respuesta del jugador:

$$B_{\text{velocidad}} = \left\lfloor \text{BONUS_VELOCIDAD_MAX} \cdot \frac{t_{\text{restante}}}{\text{TIEMPO_POR_PREGUNTA}} \right\rfloor$$

donde t_{restante} es el tiempo en segundos disponible al momento de confirmar la respuesta.

Bono de racha. El bono de racha premia al jugador por acumular respuestas correctas a lo largo de la partida. El sistema lleva un contador interno de cuántas preguntas ha respondido correctamente el jugador. Cada vez que ese contador alcanza exactamente `NUMERO_DE_RACHA`, `NUMERO_DE_RACHA×2`, `NUMERO_DE_RACHA×3`, y así sucesivamente, se suma automáticamente al puntaje acumulado un bonus de `BONUS_RACHA` puntos.

Por ejemplo, si `NUMERO_DE_RACHA= 5` y `BONUS_RACHA= 100`, y el jugador responde 10 preguntas correctas durante la partida, se suman **200 puntos** en total: `BONUS_RACHA` al acertar la quinta pregunta correcta, y otros `BONUS_RACHA` al acertar la décima. Si respondiese 15 correctas, se sumaría una tercera vez, y así sucesivamente. El contador de correctas se **reinicia a cero** al comenzar cada nueva partida. Durante el transcurso de una partida, el contador no se ve afectado por el uso de comodines a menos que el comodín sea doble oportunidad; simplemente avanza con cada respuesta correcta y se detiene cuando la partida finaliza.

3.4.2. Puntaje récord

El **puntaje récord** corresponde al máximo puntaje obtenido por el jugador en una sola partida dentro de una categoría específica. Este valor se actualiza automáticamente al finalizar cada partida si el puntaje obtenido supera el récord anterior del jugador en dicha categoría. El puntaje récord se almacena de forma independiente por categoría en la base de datos.

3.4.3. Respuesta incorrecta o tiempo agotado

Una respuesta incorrecta, o la falta de respuesta cuando el tiempo es agotado, no otorga puntaje, descuenta `PENALIZACION_INCORRECTA` puntos al jugador y **finaliza la partida**.

$$P_{\text{incorrecto}} = -\text{PENALIZACION_INCORRECTA}$$

Si `PENALIZACION_INCORRECTA = 0`, no existe descuento en puntaje, aunque la partida finaliza igualmente.

3.4.4. Derrota

El jugador sufre una derrota cuando la partida finaliza por respuesta incorrecta o tiempo agotado antes de completar todas las preguntas disponibles. En este caso, el puntaje acumulado hasta ese momento (incluyendo la penalización correspondiente) se registra igualmente en la base de datos.¹

3.5. Leaderboard

El leaderboard registra el **mejor puntaje obtenido por cada usuario** y se actualiza al finalizar cada partida. Solo las **victorias** son elegibles para el leaderboard; las partidas que terminan en derrota no se consideran. Existen dos tipos de leaderboard:

- **Global:** Muestra los `TOP_N_GLOBAL` mejores usuarios de todas las categorías, junto con el nombre del jugador, la categoría en que se obtuvo el puntaje y la fecha y hora en que se registró el resultado.
- **Por categoría:** Cada categoría cuenta con su propio ranking, mostrando los `TOP_N_CATEGORIA` mejores usuarios obtenidas específicamente en esa categoría, junto con la fecha y hora en que se registró el resultado.

¹El manejo de decimales en los cálculos de puntaje queda a criterio del estudiante (trabajar con enteros o redondear los decimales) y debe ser documentado en el `README.md` del proyecto.

4. Interfaz Gráfica e interacción

Se evaluarán, entre otros, los siguientes aspectos:

- Correcta **modularización** del programa, esto quiere decir que se debe respetar y seguir una adecuada estructuración entre *front-end* y *back-end*, con un **diseño cohesivo** y de **bajo acoplamiento**.
- Correcto uso de **señales** entre *back-end* y *front-end*, implementación de *threading* cuando corresponda.
- Un flujo prolijo a lo largo del programa. Esto quiere decir que el usuario puede navegar sin problemas entre las distintas partes que componen *DCCuestionados* solo ejecutando una vez el programa. En otras palabras, un usuario nunca debe quedarse atorado en alguna parte del programa que lo obliga a cerrar *DCCuestionados* y volver a ejecutarlo. A modo de ejemplo: si un usuario ingresa a una ventana y no se puede mover a otra ventana, esto es considerado como una mala implementación.

Aclaración importante: los esquemas y diseños que serán expuestos son únicamente referenciales. No es necesario que tu tarea sea una copia exacta de estos: puedes agregar creatividad inventando botones nuevos, interacciones nuevas, diseños y hasta animaciones. Sin embargo, será evaluado que los elementos **mínimos** estén presentes y funcionen del modo que se exige (detallado más adelante).

Estos elementos mínimos se explicitan en cada sección y hacen referencia a comportamientos esperados con la interacción, y a información que debe ser mostrada. A menos que se explicita por medio del nombre de un *widget* específico, eres libre de mostrar la información y crear la interacción por medio de los componentes que desees, ya sean inventados por ti o propios de la librería **PyQt**.

A pesar de lo anterior, en ningún caso se les exige cosas relacionadas a la “belleza” de la interfaz gráfica. Por lo tanto, una ventana que sólo tenga los elementos expuestos en los esquemas, tiene el mismo nivel de validez que otra llena de decoraciones y efectos interactivos, esto, suponiendo que ambas tengan el comportamiento deseado.

Finalmente cabe destacar que si se desea desarrollar funcionalidades extras, estas serán evaluadas bajo los mismos criterios generales mencionados anteriormente. Esto quiere decir que, si el programa falla debido a una funcionalidad extra, se aplicará el descuento en el ítem correspondiente.

4.1. Ventana de Inicio de sesión

Es la primera ventana que se debe mostrar al ejecutar el programa. Su función principal es solicitar al usuario que ingrese su nombre de usuario y su contraseña. Debe tener como mínimo los siguientes elementos:

- **Nombre del programa:** Se debe presentar como texto el nombre del juego
- **Editores de texto:** Debe tener al menos 2 editores, uno para el nombre del usuario y otro para la contraseña.
- **Botón de ingreso:** Debe tener un botón con el que se solicita el ingreso al juego.

En la siguiente figura se muestra un estilo de ventana donde aparece cada componente mínimo de la ventana (no es necesario seguir el estilo mostrado).



Figura 1: Esquema de ventana de inicio.

Además, en esta ventana se debe mostrar la información relacionada con el estado del servidor y el rechazo de inicio de sesión. Esto se puede lograr agregando un elemento de texto en la ventana, agregando un mensaje de dialogo, etc.

Tener en cuenta que si el usuario no existe, se registra al usuario como usuario nuevo e inicia sesión con ese nuevo usuario, esto se puede ver en detalle en [WebServices](#) en el POST /auth.

4.2. Ventana principal

Esta ventana se muestra si la conexión al servidor es exitosa y la validación de usuario es correcta al iniciar sesión en la ventana anterior. Las requisitos minimos son variados y se muestran a continuación:

- **Estadísticas del usuario:** Se debe poder ver el nombre del usuario además de sus estadísticas como las partidas ganadas/perdidas, el puntaje acumulado, etc. Estas deben venir de el endpoint **GET /usuarios/<username>/estadísticas** que aparece en [WebServices](#)
- **Selector de categoría para preguntas:** Hay un medio de selección de las categorías de las preguntas con las que se va a jugar, estas se deben mostrar al usuario para que pueda elegir. Para la elección se puede usar un menú desplegable, un lugar de selección multiple, etc. Tener en cuenta que las categorías no pueden estar hardcodeadas y deben obtenerse desde la [API externa](#) a través del servidor.
- **Leaderboard:** Se deben mostrar los **TOP_N_GLOBAL** mejores usuarios junto al puntaje obtenido por cada usuario para el top global. También se debe mostrar un leaderboard por cada categoría con los **TOP_N_CATEGORIA** mejores usuarios.
- **Botón de jugar:** Se tiene un botón de jugar para empezar el juego, se debe obtener la categoría de las preguntas desde el selector de categorías.
- **Botón de salir:** Se tiene un botón de salida que devuelve al usuario a la ventana de inicio de sesión y termina la sesión del usuario desconectando al jugador del servidor con el [Protocolo de Desconexión Robusta y Manejo de Errores](#).

En la siguiente figura se muestra un estilo de ventana donde aparece cada componente mínimo de la ventana (no es necesario seguir el estilo mostrado).



Figura 2: Esquema de ventana principal.

4.3. Ventana de preguntas

Al iniciar el juego se mostrará esta ventana donde se podrán responder las preguntas para la categoría elegida.

Esta ventana tiene los siguientes requisitos:

- **Contador de tiempo:** Cada pregunta tiene un tiempo limite para ser respondida, en la ventana se debe mostrar como se agota el tiempo limite, ya sea con una barra que va desapareciendo, una cuenta regresiva usando un numero, etc.
- **Pregunta y respuestas:** Se debe mostrar la pregunta a responder junto con las posibles respuestas y alguna forma para que el usuario responda esa pregunta. Las respuestas mostradas deben ser ordenadas al azar.
- **Puntaje actual:** Se debe mostrar el puntaje actual que lleva el usuario en su partida y se debe ir actualizando cada vez que el usuario responde una pregunta.
- **Puntaje récord del usuario:** Se debe mostrar el puntaje récord histórico del usuario en la categoría elegida. Este valor solo se actualiza después de terminar la partida.
- **Comodines:** Se deben mostrar los comodines que se pueden usar para la pregunta actual y se deben poder usar en mitad de la pregunta para activar el efecto. Si se usa el comodín se debe mostrar su efecto en la ventana y debe quedar claro que no se puede usar una segunda vez (desactivando el botón, eliminando el botón, etc). Además, si hay comodines excluidos por el comodín activado, se deben desactivar pero solo por la pregunta actual. La exclusión de comodines se puede ver en la tabla de la sección [Comodines](#).

En la siguiente figura se muestra un estilo de ventana donde aparece cada componente mínimo de la ventana (no es necesario seguir el estilo mostrado).

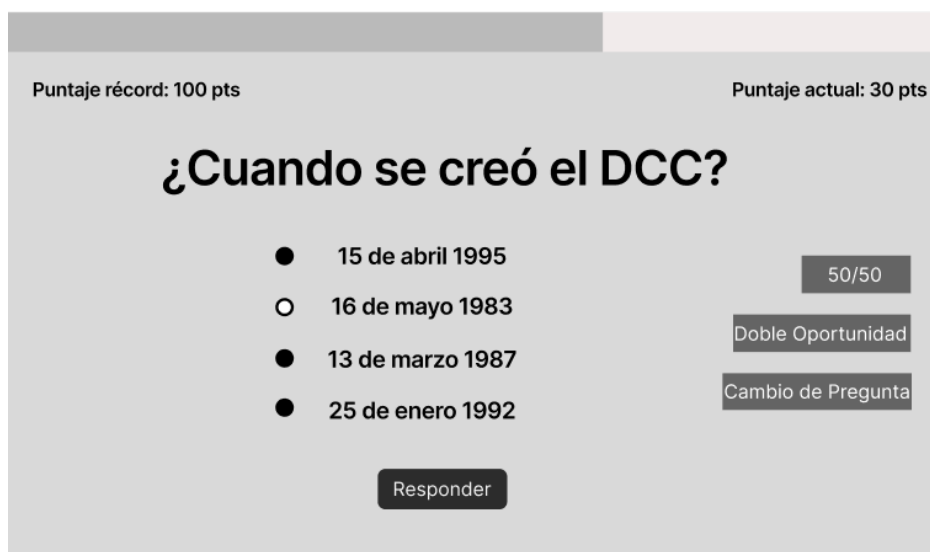


Figura 3: Esquema de ventana de preguntas.

Tomar en cuenta que el contador de tiempo en el esquema se representa con la barra superior y va desapareciendo cada segundo.

4.4. Ventana de Fin de partida

Esta ventana se muestra cuando se termine una partida, su funcionalidad principal es mostrarle al usuario los datos de su partida ya finalizada. Esta ventana tiene los siguientes requisitos:

- **Puntaje total:** Se debe mostrar el puntaje total obtenido en la partida.
- **Numero de preguntas respondidas correctamente:** Se debe mostrar el numero de preguntas respondidas correctamente.
- **Duración de la partida:** Se debe mostrar la duración total de la partida en formato “min : seg”.
- **Botón de volver al inicio:** Debe tener un botón que al presionarlo el usuario se devuelva a la ventana principal.

En la siguiente figura se muestra un estilo de ventana donde aparece cada componente mínimo de la ventana (no es necesario seguir el estilo mostrado).



Figura 4: Esquema de ventana de fin de partida.

5. Arquitectura cliente-servidor

El sistema se estructurará bajo un modelo *Cliente-Servidor* distribuido. El *cliente* actuará como la interfaz gráfica e interactiva del usuario, mientras que el servidor central operará como un *proxy* del sistema, abstrayendo el consumo de una API externa de preguntas y gestionando una API interna mediante sockets TCP con API, manejados con serialización por *pickle*, específicos para la arquitectura cliente-servidor y endpoints HTTP para la sincronización y persistencia de los datos, exclusivo para uso interno del servidor *Flask*.



Figura 5: Diagrama de la arquitectura

5.1. Serialización

Se implementará un protocolo de serialización para la comunicación cliente-servidor a través de sockets TCP. Para que el receptor pueda decodificar la información de manera eficiente y sin ambigüedades, cada *payload* de bytes transmitido por la red se estructurará mediante un encabezado fijo de tamaños al inicio, seguido por los datos variables:

- **Encabezado Fijo (Header - 12 bytes en total):**
 - **Largo del Token (4 bytes):** Un entero que indica la longitud exacta en bytes del token de autenticación.
 - **Largo del Username (4 bytes):** Un entero que indica la longitud en bytes del nombre de usuario.

- **Largo del Mensaje (4 bytes):** Un entero que indica la longitud en bytes del mensaje serializado con *pickle*.
- **Bloque de Datos Variables (Body - Largo dinámico):**
 - **Token:** La cadena de autenticación del usuario. Para más información, ver `POST /auth` en [WebServices](#)). Este debe compararse con el `token` guardado en el servidor para proceder con cualquier petición.
 - **Username:** El nombre del jugador codificado en formato *UTF-8*.
 - **Mensaje:** El contenido o comando específico de la interacción, serializado mediante la librería *pickle*.

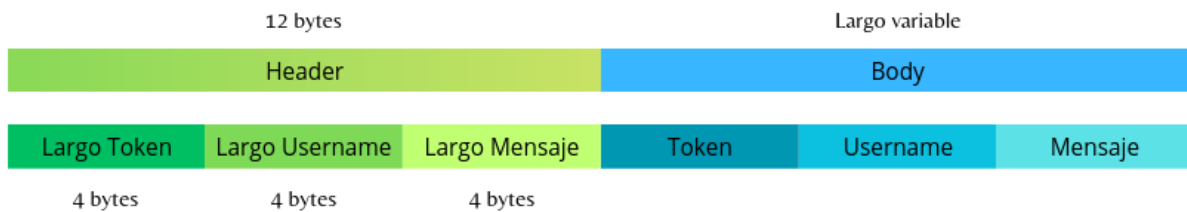


Figura 6: Diagrama de segmentación del *bytearray*

Al recibir el paquete, el servidor leerá secuencialmente los primeros 12 bytes fijos. Con estos tres enteros, el backend sabrá con precisión cuántos bytes segmentar de la carga útil para reconstruir de forma limpia cada uno de los parámetros del jugador.

5.2. API externa

5.2.1. Elección y requisitos de la API externa

El servidor debe consumir una API pública externa de trivia para abastecer el juego con preguntas. La elección de la API queda a **criterio del estudiante**, siempre que sea **pública** (sin requerir de cuenta) y permita obtener preguntas con sus alternativas, la respuesta correcta y la categoría asociada.

Ejemplos de APIs que cumplen estos requisitos son [Open Trivia DB](#) y [The Trivia API](#), pero cualquier otra equivalente es válida.

5.2.2. Documentación obligatoria en el README

En el archivo `README.md` de la tarea debes documentar, como mínimo:

- El **nombre** de la API elegida y su **URL base**.
- Los *endpoints* específicos que consume tu servidor, junto con un ejemplo de petición real (por ejemplo: `GET https://opentdb.com/api.php?amount=50&category=18`).
- Los *query params* relevantes y una breve descripción de cada uno.
- La **estructura del JSON** que tu servidor espera recibir, idealmente acompañada de un ejemplo abreviado.

Esta documentación es **obligatoria** y será revisada durante la corrección. Sin ella, no será posible verificar que el servidor consume correctamente la API externa.

Importante: únicamente el servidor debe consumir esta API; el cliente **nunca** debe consultarla directamente.

5.2.3. Política de consumo y almacenamiento local

Para optimizar el uso de la red y evitar el bloqueo por exceso de peticiones (*rate limiting*), el servidor no consultará la API externa por cada pregunta individual que solicite un jugador. En su lugar, el sistema mantendrá un registro de preguntas en su base de datos local, el cual se actualizará mediante consultas por bloques (*batches*).

Este almacenamiento local se refrescará de forma híbrida: se realizará una carga masiva inicial al arrancar el servidor y, posteriormente, se activará una actualización por bloques cada vez que el inventario de preguntas disponibles de la categoría en la base de datos local caiga por debajo de un umbral mínimo del 20 %, pudiendo dar la misma pregunta a más de un usuario. Esto asegura la disponibilidad inmediata de contenido sin saturar la API externa.

5.3. Concurrencia y Gestión de Conexiones

El servidor debe estar diseñado para soportar la concurrencia de múltiples clientes simultáneos en tiempo real, garantizando que el flujo de peticiones no bloquee el hilo principal de ejecución ni afecte el rendimiento de otras partidas en curso.

5.3.1. Multi-threading

- **Delegación de Conexiones:** Cada vez que el socket del servidor acepte una nueva conexión, esta debe ser delegada inmediatamente a un hilo independiente. El hilo principal del servidor debe quedar liberado exclusivamente para continuar escuchando nuevas solicitudes de acceso en el puerto.
- **Encapsulamiento del Estado:** Cada sesión de juego debe operar de forma estrictamente aislada. Las variables críticas de la partida (identificador de usuario, preguntas respondidas, puntaje acumulado y temporizadores) deben estar encapsuladas en instancias de clase independientes por cada hilo.

5.3.2. Sincronización y Seguridad de Hilos

Dado que existen recursos compartidos a los que acceden múltiples hilos de manera simultánea (el almacenamiento local de preguntas, el archivo de persistencia de datos o la estructura del *leaderboard*), se exige el uso explícito de manejo de zonas críticas para la sincronización. Esto es mandatorio para prevenir condiciones de carrera (*race conditions*) durante las operaciones de lectura y escritura concurrentes.

5.3.3. Protocolo de Desconexión Robusta y Manejo de Errores

El sistema de backend debe ser tolerante a fallos y gestionar el ciclo de vida de los sockets de manera limpia:

- **Detección de Interrupciones:** El servidor debe ser capaz de capturar excepciones o descodificaciones fallidas de *pickle* que ocurran cuando un cliente cierre la ventana de PyQt6 de forma abrupta o sufra una pérdida de conectividad.
- **Liberación de Recursos:** Al confirmarse la desconexión de un jugador, el servidor deberá ejecutar un protocolo de limpieza que incluya: cerrar el socket asignado de forma segura y eliminar al usuario del registro de clientes activos.
- **Caída del servidor:** En caso de que el servidor sufra alguna interrupción, el cliente debe ser capaz de detectar la situación, volver al menú principal e informar al jugador de la caída del servidor.

Tanto para la detección de interrupciones como la liberación de recursos requieren *logs* detallados por consola. Es obligatorio poner el **Username** en los *logs* por consola del servidor.

Un ejemplo de *log* detallado puede ser el siguiente:

```
[LOG] Jugador agustinbecker9 sufrió una desconexión inesperada.
```

aunque este formato no es obligatorio.

6. *WebServices*

Cuando se inicia el servidor, además de abrir los puertos necesarios para que los clientes se conecten, deberá levantar una API que ofrezca diferentes *endpoints* para gestionar el flujo del juego. Es fundamental recordar que esta API actúa como el único intermediario entre el cliente, la base de datos interna, y la API externa de trivia. El cliente nunca debe tener comunicación directa con la base de datos ni con la API externa.

Algunos de estos *endpoints* estarán disponibles públicamente sin la necesidad de autenticación, mientras que otros requerirán el uso de un *token* de autenticación para validar la acción asociada. Para la autenticación, se debe ocupar el *header* de la *request* usando el valor de `TOKEN_AUTENTICACION` que debe ser retornado y guardado tanto en servidor y en el cliente para poder verificar que la procedencia de la *request* es de un cliente previamente autenticado.

Esta API debe levantarse en el *host* y puerto definidos en los parámetros de conexión, y deberá estar disponible en todo momento mientras el servidor se esté ejecutando. No se debe crear un archivo de entrada (`main.py`) distinto para esta API, sino que el servidor debe ser capaz de ofrecer los *endpoints* ejecutando solo un `main.py`, el cual además se encarga de toda la lógica central del juego. Para lograr esto de forma concurrente, deberás usar **Threads**.

Este WebService es la entidad exclusiva que lee y modifica la información de los archivos en la Base de datos, y quien realiza las peticiones hacia la API externa para obtener las categorías y los *batches* de preguntas. Por otro lado, el retorno de cada *endpoint* debe ser del tipo `flask.Response`, asegurando retornar siempre el código de estado HTTP adecuado.

Los *endpoints* mínimos que debe tener esta API se describen a continuación. En este listado, si un *endpoint* se indica como (*privado*), deberás integrar la validación del *token* necesaria para asegurar que **solo** el servidor sea capaz de interactuar con este. Recuerda que la API Flask es de uso estrictamente interno para el servidor. El cliente nunca debe realizar peticiones HTTP a la API. Cuando el servidor necesite leer o escribir en la base de datos, deberá realizar una petición HTTP interna a la API Flask (la que el mismo servidor levantó), adjuntando en el header el `TOKEN_AUTENTICACION`.

(público) POST /auth Este *endpoint* maneja el flujo unificado de inicio de sesión y registro de jugadores. Valida si el usuario existe y si su contraseña es correcta, o lo registra de forma automática en caso de no existir y que el nombre de usuario cumpla con ser alfanumérico (no tener comas ni espacios), asociándole la contraseña ingresada.

Header: No debe incluir información de autenticación.

Body: Debe contener el `username` y la `password` del jugador en formato JSON.

⇒ Retorna: Un código de éxito y su `TOKEN_AUTENTICACION` si la validación es correcta. Retorna un código de error si la contraseña es incorrecta, o si el usuario ya posee una sesión activa en el servidor.

(público) GET /categorias Este *endpoint* retorna el listado completo de las temáticas o categorías de trivia disponibles en el sistema para ser desplegadas en el *Lobby*.

Header: No debe incluir información de autenticación.

Body: No debe incluir.

⇒ Retorna: Un código de éxito junto con un JSON con el arreglo de categorías (con sus identificadores y nombres), o un código de error, según corresponda.

(privado) GET /preguntas Permite al cliente obtener un lote de preguntas asociadas a una categoría específica para abastecer la partida. El servidor enviará automáticamente `NUMERO_PREGUNTAS + 1` preguntas (para soportar el comodín de cambio de pregunta). *Importante: El lote enviado al cliente no debe contener el indicador de la respuesta correcta, de lo contrario, el sistema sería vulnerable a trampas.*

Header: Debe incluir el *token* de autenticación.

Query Params: Debe incluir `categoria_id`.

Body: No debe incluir.

⇒ Retorna: Un código de éxito, junto con un JSON que contiene el `partida_id` generado para la partida y la lista de las preguntas. Cada pregunta de la lista debe incluir su `pregunta_id`, el texto de la pregunta, y sus alternativas correspondientes, ordenadas al azar y sin incluir la respuesta correcta. Retorna un código de error apropiado si la categoría no existe.

(privado) POST /preguntas/verificar Verifica de manera inmediata si la alternativa seleccionada por un jugador para una determinada pregunta es la correcta, calcula el puntaje obtenido en esa jugada y actualiza el estado de la partida en el servidor.

Header: Debe incluir el *token* de autenticación.

Body: Debe incluir el `pregunta_id`, la `respuesta_usuario`, y el `partida_id`.

⇒ Retorna: Un código de éxito, junto con un JSON indicando si la respuesta es correcta o no, el `puntaje_ganado` en esa pregunta específica, y el `puntaje_actual` que lleva el usuario en la partida actualizado.

(privado) POST /comodines/activar Registra la activación de un comodín en el servidor para la pregunta en curso y devuelve la información específica necesaria para que el cliente actualice la interfaz gráfica, todo esto antes de que el usuario confirme y envíe su respuesta definitiva.

Header: Debe incluir el *token* de autenticación.

Body: Debe incluir el `partida_id`, el `pregunta_id` actual y un identificador del comodín usado en formato JSON (ustedes mismos definen qué usar como identificador del comodín, no es algo que venga de la API).

⇒ Retorna: Un código de éxito, junto con un JSON que dependerá del comodín activado:

- Si es 50/50: Retorna una lista con las alternativas a eliminar para que el front-end las desactive u oculte visualmente.
- Si es Cambio de Pregunta: Retorna un objeto con la nueva pregunta, incluyendo su respectivo `pregunta_id`, el texto de la pregunta y las alternativas ordenadas al azar (sin indicar la respuesta correcta).

- Si es Doble Oportunidad: Retorna un mensaje de confirmación de activación para que el cliente sepa que el servidor ya autorizó la regla del doble intento para esa `pregunta_id`

O retorna un código de error apropiado si el comodín ya fue utilizado en la partida, o si la combinación de comodines activados simultáneamente es inválida.

(privado) POST /puntajes Finaliza la partida actual del usuario, calcula el puntaje final obtenido basándose en el desempeño registrado internamente en el servidor y actualiza su historial de forma persistente.

Header: Debe incluir el *token* de autenticación.

Body: Debe incluir el `username`, el `categoria_id`, y el `partida_id`.

⇒ Retorna: Un código de éxito, junto con un JSON que contenga el `puntaje_final` calculado por el servidor y la cantidad de `respuestas_correctas`, para que el cliente pueda mostrarlos en la ventana de fin de partida.

(privado) GET /usuarios/<username>/record Consulta el puntaje máximo histórico que ha obtenido un usuario específico en una categoría determinada.

Header: Debe incluir el *token* de autenticación.

Query Params: Debe incluir el `categoria_id`.

Body: No debe incluir.

⇒ Retorna: Un código de éxito, junto con un JSON con el puntaje máximo registrado para ese usuario. Si no hay registros, retorna un código de error apropiado.

(público) GET /leaderboards Obtiene los puntajes más altos del juego para estructurar el ranking global o por categoría que se visualiza en la Ventana Principal. El resultado retornado debe respetar estrictamente el límite máximo de usuarios configurado en el sistema.

Header: No debe incluir información de autenticación.

Query Params: Puede incluir `categoria_id` de forma opcional para filtrar los puntajes por una temática específica.

Body: No debe incluir.

⇒ Retorna: Un código de éxito, junto con un JSON. Si se provee un `categoria_id`, retorna un arreglo ordenado descendientemente con los mejores puntajes de esa categoría. Si no se provee un `categoria_id`, debe retornar en una única respuesta tanto el arreglo del ranking global como los rankings individuales de cada categoría disponible (esto permite al cliente cargar toda la información inicial del *Lobby* ejecutando una única petición).

(privado) GET /usuarios/<username>/estadisticas Permite al cliente obtener el historial completo y las estadísticas acumuladas de un jugador para ser desplegadas en la interfaz gráfica.

Header: Debe incluir el *token* de autenticación.

Query Params: No debe incluir.

Body: No debe incluir.

⇒ Retorna: Un código de éxito, junto con un JSON con el `puntaje_acumulado_total`, la cantidad de victorias, la cantidad de derrotas, el total de `partidas_jugadas` y un arreglo con el puntaje acumulado por cada `categoria_id`.

7. Base de datos

En esta sección se describen los archivos que se deben utilizar para registrar la información del juego y los jugadores. Estos archivos son leídos y modificados exclusivamente a través de los *WebServices* que ofrece el servidor, se interactúa con ellos únicamente a través de la **API**. En caso de que alguno de estos archivos no exista al iniciar el servidor, el programa debe ser capaz de crearlo desde cero y rellenarlo correctamente.

Los archivos mínimos que debe manejar la base de datos son:

1. **usuarios.json**: Este archivo almacena el perfil completo, las credenciales y el historial acumulado de cada jugador registrado. Se recomienda estructurarlo como un diccionario donde la llave principal sea el **username**. Para cada usuario, se debe guardar su contraseña (estrictamente su versión *hasheada*), el puntaje acumulado total, la cantidad de victorias, las derrotas, el total de partidas jugadas, un sub-diccionario con el puntaje acumulado por cada categoría jugada, y un sub-diccionario con el récord histórico por cada categoría jugada.

Para el *hashing* de la contraseña, se deberá utilizar el algoritmo **SHA-256 sin salt**, que viene implementado en la librería estándar `hashlib`². El algoritmo opera sobre bloques de datos binarios, por lo que será necesario codificar la contraseña (un *string*) a *bytes* antes de pasarla como argumento a la función de encriptación `hashlib.sha256()`.

Un ejemplo de la estructura esperada es:

```
1 {
2   "Ag3ntCooper": {
3     "password": "5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1",
4     "puntaje_acumulado_total": 4500,
5     "victorias": 12,
6     "derrotas": 3,
7     "partidas_jugadas": 15,
8     "puntaje_por_categoria": {
9       "1": 2000,
10      "3": 2500
11    },
12    "record_por_categoria": {
13      "1": 5000,
14      "3": 2500
15    }
16  }, ...
17 }
```

²Para más información, consultar la [documentación oficial de hashlib](#)

2. `partidas.csv`: Este archivo funciona como un registro histórico de todas las partidas jugadas. Cada vez que finaliza una partida (ya sea por victoria o derrota), el servidor debe añadir una nueva línea a este archivo. La información contenida en cada línea debe ser, en orden, y separada por comas sin espacios: el identificador de la partida (que debe ser un contador numérico autoincremental), el `username` del jugador, el `categoria_id` de la trivia jugada, el puntaje final obtenido, la cantidad de respuestas correctas, un indicador de si fue Victoria o Derrota, y finalmente el *timestamp* de término de la partida en formato *Unix time*.

Este archivo es útil para que el servidor pueda calcular rápidamente los *Leaderboards* globales y por categoría. Un ejemplo de su contenido:

```
1 1,SoaLedy,1,1500,10,Victoria,1761256072.6645644
2 2,XxJuanxX00,2,450,6,Derrota,1759243917.132011
3 3,Bodoqu3,3,2500,5,Victoria,1762248510.0003
4 4,XxJuanxX00,2,750,10,Victoria,1761259962.974682
```

8. Archivos

8.1. `parametros.py`

Para esta tarea, se requiere la creación de un archivo `parametros.py` tanto en la carpeta del cliente/ como en la del servidor/. En este deberás **completar todos los parámetros mencionados a lo largo del enunciado**. Dichos parámetros se presentarán en [ESTE_FORMATO](#) y en ese color. Además, es fundamental incluir cualquier valor constante necesario en tu tarea, así como cualquier tipo de *path* utilizado.

Un parámetro siempre tiene que estar nombrado de acuerdo a la función que cumplen, por lo tanto, asegúrate de que sean descriptivos y reconocibles. Un ejemplo de parametrizaciones es la siguiente:

```
1 CINCO = 5 # mal parámetro
2 PROBABILIDAD_CORRUPCION = 0.2 # buen parámetro
```

Si necesitas agregar algún parámetro que varíe de acuerdo a otros parámetros, una correcta parametrización sería la siguiente:

```
1 PI = 3.14
2 RADIO_CIRCUNFERENCIA = 3
3 AREA_CIRCUNFERENCIA = PI * (RADIO_CIRCUNFERENCIA ** 2)
```

Dentro del archivo `parametros.py`, es obligatorio que hagas uso de todos los parámetros almacenados y los importes correctamente. Cualquier información no relacionada con parámetros almacenada en este archivo resultará en una penalización en tu nota. Recuerda que no se permite el *hard-coding*³, ya que esta práctica se considera incorrecta y su uso conllevará una reducción en tu calificación.

³*Hard-coding* es la práctica de ingresar valores directamente en el código fuente del programa en lugar de parametrizar desde fuentes externas.

9. .gitignore

Para esta tarea **deberás utilizar un .gitignore** para ignorar los archivos indicados, este deberá estar dentro de tu carpeta Tareas/T3/.

Los elementos que no debes subir y **debes ignorar mediante el archivo .gitignore** para esta tarea son:

- El enunciado.
- El archivo README_inicial.md (no confundir con el archivo **obligatorio** README.md)
- Cualquier archivo *bytecode*, es decir, cualquier archivo con extensión .pyc
- Cualquier carpeta que almacene archivos de tipo dato o imágenes que no sean los entregados junto al enunciado de la tarea (los especificados en esta parte [Archivos](#))

Recuerda **no ignorar archivos vitales de tu tarea como los que tú creas o modificas, o tu tarea no podrá ser revisada.**

Es importante que hagan un correcto uso del archivo .gitignore, es decir, los archivos **deben** no subirse al repositorio debido al uso correcto del archivo .gitignore y no debido a otros medios.

10. Importante: Corrección de la tarea

Es fundamental que la tarea sea funcional, es decir, **solo se corregirán tareas que se puedan ejecutar**. Por lo tanto, se recomienda hacer periódicamente pruebas de ejecución de su tarea y *push* en sus repositorios. En el [siguiente enlace](#) se encuentra la distribución de puntajes.

Importante: Todo ítem corregido por el cuerpo docente será evaluado únicamente de forma ternaria: cumple totalmente el ítem, cumple parcialmente o no cumple con lo mínimo esperado. Finalmente, todos los descuentos serán asignados manualmente por el cuerpo docente respetando lo expuesto en [el documento de bases generales](#).

Para terminar, si durante la realización de tu tarea se te presenta algún problema o situación que pueda afectar tu rendimiento, no dudes en contactar al ayudante de Bienestar de tu sección. El correo está en el [siguiente enlace](#).

11. Restricciones y alcances

- Esta tarea es **estrictamente individual**, y está regida por el [Código de honor de Ingeniería](#).
- Tu programa debe ser desarrollado en Python 3.12.X con X mayor o igual a 9.
- Tu programa debe estar compuesto por uno o más archivos de extensión `.py` que estén correctamente ordenados por carpeta. **No se revisará archivos en otra extensión como `.ipynb`.**
- Todo el código entregado debe estar contenido en la carpeta y rama (*branch*) indicadas al inicio del enunciado. Ante cualquier problema relacionado a esto, es decir, una carpeta distinta a `Tareas/T3/` o una rama distinta a `main`, se recomienda preguntar en las [discussions del foro](#).
- Si no se encuentra especificado en el enunciado, supón que el uso de cualquier librería Python está prohibida. Pregunta en la *discussion* especial del [foro](#) si es que es posible utilizar alguna librería en particular.
- Debes adjuntar un **único archivo markdown**, llamado `README.md`, **conciso y claro**, donde describas los alcances de tu programa, cómo correrlo, las librerías usadas, los supuestos hechos, y las referencias a código externo. El no incluir este archivo, incluir un `readme` vacío o el subir más de un archivo `.md`, conllevará un [descuento](#) en tu nota.
- Esta tarea se debe desarrollar **exclusivamente** con los contenidos liberados al momento de publicar el enunciado. No se permitirá utilizar contenidos que se vean posterior a la publicación de esta evaluación.
- Se encuentra estrictamente prohibido citar código que haya sido publicado **después de la liberación del enunciado**. En otras palabras, solo se permite citar contenido que ya exista previo a la publicación del enunciado. Además, se encuentra estrictamente prohibido el uso de herramientas generadoras de código para el apoyo de la evaluación.
- Cualquier aspecto no especificado queda a tu criterio, siempre que no pase por sobre otro que sí sea especificado por enunciado.

Las tareas que no cumplan con las restricciones del enunciado obtendrán la calificación mínima (1,0).